

I116 SISTEMAS DISTRIBUIDOS Y PARALELOS

Trabajo Práctico - Régimen de Promoción

Paralelización del problema de las N-ReinasGrupo 18Avila Montoya, Eygleen Fernanda¹ (02931/2)Bejarano, Abril¹ (03339/5)

27/05/2026

¹{eygleen.avila, abril.bejarano}@alu.ing.unlp.edu.ar**1. Descripción del algoritmo secuencial**

El algoritmo secuencial implementa una solución para el problema de las N-Reinas utilizando Backtracking y operaciones a nivel de bits para optimizar los cálculos.

1.1. Representación de los elementos

Se representa el tablero utilizando un arreglo de enteros donde cada índice del arreglo representa una fila del tablero y el valor almacenado en cada posición se interpreta como una máscara de bits. En cada número entero, un bit en 1 indica la columna donde se encuentra la reina de esa fila, mientras que los bits en 0 representan las casillas vacías.

1.2. Estrategia para determinar si un tablero es válido

El algoritmo construye las soluciones garantizando que cada nueva reina agregada esté en una posición segura. Para esto, mantiene las variables down, left y right, que se actualizan en cada llamada a las funciones recursivas para tener un registro de las posiciones que están amenazadas por las reinas de las filas anteriores y, por lo tanto, son inválidas.

Las posiciones válidas para la fila actual se marcan con un 1 en la variable bitmap, la cual se calcula a partir de las variables down, left y right.

Además de generar únicamente tableros que cumplen las condiciones del problema de las N-Reinas, también se descartan todos los tableros que sean rotaciones o espejos de soluciones ya encontradas. Esto se logra limitando las columnas iniciales evaluadas en la primera fila (con variables como BOUND1) y utilizando la función Check() para reconocer simetrías.

1.3. Almacenamiento de resultados y contabilización

El arreglo BOARD se sobrescribe continuamente a medida que el algoritmo se ejecuta. Para llevar el registro de los tableros válidos, se utilizan contadores según la simetría de la solución hallada.

Si una solución puede rotarse 90, 180 y 270 grados, luego espejarse y realizar las mismas rotaciones, y si las ocho configuraciones resultantes son diferentes, entonces se suma a COUNT8. Puede suceder que alguna de estas rotaciones resulte en un tablero idéntico al que se tiene, y es en estos casos que se utilizan COUNT4 y COUNT2, según el nivel de simetría.

Finalmente, el número total de tableros válidos se calcula multiplicando cada contador por la cantidad de tableros que representa ($TOTAL = COUNT8 * 8 + COUNT4 * 4 + COUNT2 * 2$), mientras que el total de tableros únicos se obtiene simplemente sumando los contadores ($UNIQUE = COUNT8 + COUNT4 + COUNT2$).

2. Estrategia y descripción de las etapas de diseño paralelo

2.1. Descomposición

La estrategia de descomposición más adecuada es la de **descomposición exploratoria**, ya que la estructura del problema no se conoce completamente al inicio y evoluciona durante la ejecución, explorando un espacio de soluciones posibles. La descomposición progresa dinámicamente: a medida que se explora el espacio de búsqueda se generan nuevas tareas, lo que hace que el paralelismo no esté completamente definido desde el comienzo.

En este caso, se optó por que el proceso con rank 0 realice las primeras iteraciones del algoritmo, creando tareas que luego los otros procesos puedan tomar y distribuir entre sus hilos. Esto se hace para balancear mejor la carga, ya que cada tablero puede requerir una cantidad de operaciones muy diferente. Una tarea en este caso es, entonces, uno de esos tableros intermedios.

Todos los tableros son independientes entre sí, por lo que no hay dependencias entre las tareas generadas.

2.2. Comunicación

En este problema, cada tarea representa un subárbol del backtracking y, una vez que recibe su estado inicial, puede ejecutarse de manera independiente. Por eso, la comunicación no ocurre durante el cómputo interno de cada tarea, sino principalmente para distribuir trabajo y devolver resultados parciales.

La comunicación se organiza en dos niveles, siguiendo el modelo híbrido MPI + Pthreads. A nivel inter-nodo se utiliza MPI, mediante una comunicación explícita por pasaje de mensajes, punto a punto, centralizada y dinámica bajo demanda. El rank 0 actúa como master, administra el pool global de tareas y entrega lotes a los ranks trabajadores cuando estos los solicitan. Cuando no quedan tareas, envía un lote vacío como señal de finalización, y al terminar cada rank trabajador devuelve sus contadores parciales.

A nivel intra-nodo se utiliza Pthreads, aprovechando la memoria compartida entre los hilos de un mismo proceso. Los hilos toman tareas desde estructuras compartidas protegidas con mutexes y, en los ranks remotos, se usan variables de condición para avisar la llegada y finalización de lotes. Durante el procesamiento de cada tarea no hay comunicación entre hilos, ya que cada uno trabaja con su propio tablero y sus propios contadores parciales, reduciendo condiciones de carrera, sincronización innecesaria y contención.

2.3. Aglomeración

La decisión principal de aglomeración fue generar tareas a una profundidad fija ($y=3$) del árbol de búsqueda y almacenarlas en un pool global. Esto evita una granularidad excesivamente fina, donde cada colocación de reina implicaría comunicación o sincronización, y también evita una granularidad demasiado gruesa, donde pocos subárboles grandes podrían producir desbalance.

Cada tarea conserva el estado necesario para continuar el backtracking. Además, para los procesos remotos, las tareas no se envían de a una sino agrupadas en lotes de tamaño máximo BATCH_SIZE, reduciendo la cantidad de mensajes MPI y balanceando el costo de arranque de cada comunicación.

Esta aglomeración también favorece la localidad y reduce la sincronización dentro de cada nodo. Cada hilo procesa una tarea completa usando su propio estado local de tablero y contadores parciales, por lo que durante la exploración del subárbol no necesita actualizar continuamente variables globales compartidas. Los resultados se acumulan localmente y recién se combinan al finalizar, reduciendo contención y accesos compartidos. En los ranks remotos, el proceso recibe un lote por MPI y luego lo reparte entre sus hilos mediante Pthreads, manteniendo el esquema híbrido.

Por lo tanto, la aglomeración elegida establece un compromiso entre overhead y balance de carga. Los lotes permiten disminuir la frecuencia de comunicación entre procesos, mientras que la existencia de múltiples tareas en el pool permite que la asignación posterior siga siendo dinámica. Esto es importante en N-Reinas porque distintos subárboles del backtracking pueden tener costos muy diferentes.

2.4. Mapeo

En esta implementación se utilizaron dos estrategias de **mapeo dinámico centralizado**. A nivel de nodos, el proceso con rank 0 es el encargado de generar las tareas y distribuirlas entre los demás nodos. Cada vez que un nodo finaliza el conjunto de tareas asignadas, le solicita nuevas tareas y este entrega

más trabajo mientras queden tareas disponibles. Esto corresponde a un mapeo **Master-Worker**. Dentro de cada nodo, el mapeo entre los hilos se realiza con una estrategia **Bag of Tasks**. En los hilos que corresponden a los nodos worker, las tareas recibidas se almacenan en un vector compartido local, desde el cual los distintos hilos extraen trabajo hasta que se vacíe. Una vez que todas las tareas locales fueron procesadas, el nodo vuelve a solicitar nuevas tareas. Los hilos del nodo Master, en cambio, toman tareas directamente del task pool. Esto es posible por la arquitectura de memoria compartida que tiene internamente cada nodo. De esta manera se evita la necesidad de pedir tareas que ya se encuentran en su espacio de memoria.

Combinando estas dos estrategias se logra un mejor balance de carga, ya que las unidades de procesamiento que terminan antes continúan obteniendo trabajo mientras existan tareas pendientes.

Además, el uso de vectores locales reduce la frecuencia de comunicación con el master, disminuyendo en parte el overhead de coordinación global. Esto permite aprovechar mejor los recursos compartidos dentro de cada nodo y reduce el tiempo en el que las unidades de procesamiento permanecen ociosas.

3. Tiempos de ejecución, métricas y análisis de escalabilidad

3.1. Análisis de tiempos de ejecución

Se obtuvieron los siguientes tiempos de ejecución. Para $UP=1$ se muestran los T_s , resultado de ejecutar el algoritmo secuencial. Para el resto, se muestra $T_p(P)$, resultado de ejecutar el algoritmo paralelo desarrollado con la cantidad de hilos necesaria.

Tiempo de ejecución (s)	CARGA (N)				
UP	14	15	16	17	18
1	0.180799	1.114150	7.252414	51.850495	364.626777
4	0.077322	0.407687	2.589023	17.822687	129.300311
8	0.041199	0.184199	1.132308	7.703234	55.764344
16	0.029512	0.096900	0.546728	3.676072	26.567920

Para los tamaños de tableros menores, el algoritmo paralelo ya muestra mejoras respecto de la versión secuencial, aunque el beneficio obtenido es relativamente bajo.

A medida que aumenta el tamaño del problema, el paralelismo se aprovecha mejor. En los casos con 8 y 16 unidades de procesamiento, los tiempos de ejecución disminuyen considerablemente respecto de la versión secuencial.

3.2. Análisis de speedup

Se calculó la tabla de speedups a partir de los tiempos de ejecución de la siguiente manera:

$$S(P) = \frac{T_s}{T_p(P)}$$

Speedup	CARGA (N)				
UP	14	15	16	17	18
1	1	1	1	1	1
4	2.34	2.73	2.80	2.91	2.82
8	4.39	6.05	6.40	6.73	6.54
16	6.13	11.50	13.26	14.11	13.72

Para todos los escenarios evaluados, el speedup obtenido es mayor que 1, lo que indica que el algoritmo paralelo logra mejorar el rendimiento respecto de la versión secuencial incluso para los tamaños de problema más chicos. Sin embargo, para cargas menores, las mejoras observadas son moderadas y el speedup se encuentra bastante alejado del ideal. Para $N=14$ se observa que, si bien el tiempo de ejecución mejoraba, no es una mejora significativa ya que el speedup no se acerca al ideal. Esto se debe a que, para tamaños de problema chicos, el overhead de comunicación sigue siendo significativo frente al tiempo de

cómputo.

A medida que aumenta el tamaño del problema, y para una cantidad fija de unidades de procesamiento, el speedup mejora considerablemente y comienza a acercarse al valor ideal. Este comportamiento concuerda con la ley de Gustafson-Barsis. Para una cantidad fija de unidades de procesamiento, al incrementar el tamaño del problema el speedup crece y se aproxima cada vez más al ideal. Esto puede observarse claramente en las ejecuciones con 16 unidades de procesamiento, donde el speedup es cada vez más significativo a medida que aumenta la carga de trabajo.

Para las cargas analizadas, los límites impuestos por la ley de Amdahl no se perciben. Si observamos cualquier columna de la tabla, no se ve un estancamiento en el valor del speedup. Esto significa que podemos agregar más unidades de procesamiento, ya que todavía podríamos mejorar el tiempo de ejecución.

3.3. Análisis de escalabilidad

Se calculó la tabla de eficiencia a partir de la tabla de speedup, para poder analizar la escalabilidad del algoritmo. Para esto se usó la relación:

$$E(P) = \frac{S(P)}{P}$$

Eficiencia	CARGA (N)				
UP	14	15	16	17	18
1	1	1	1	1	1
4	0.59	0.68	0.70	0.73	0.70
8	0.55	0.76	0.80	0.84	0.82
16	0.38	0.72	0.83	0.88	0.86

Cuando analizamos N=14 podemos ver que, menores tiempos de ejecución y un mayor speedup al aumentar la cantidad de unidades de procesamiento, en realidad se están aprovechando cada vez menos los recursos. La paralelización sí aporta beneficios, pero implica un consumo innecesario de recursos y energía.

Para el resto de las cargas, se puede analizar escalabilidad fuerte y escalabilidad débil.

Un programa paralelo es **fuertemente escalable** si la eficiencia se mantiene aproximadamente constante al incrementar el número de unidades de procesamiento sin aumentar el tamaño del problema. Para analizar esta escalabilidad entonces, miramos las columnas de la tabla. En todos los casos para N entre 15 y 18 se puede observar que la eficiencia se mantiene aproximadamente constante, por lo que el algoritmo es fuertemente escalable para estas cargas.

Un programa paralelo es **débilmente escalable** si la eficiencia se mantiene aproximadamente constante al incrementar simultáneamente el número de unidades de procesamiento y el tamaño del problema. Por lo tanto, observamos las diagonales de la tabla. Si observamos la diagonal que va desde N=16 a N=18, vemos que la eficiencia se mantiene aproximadamente constante (0,70 - 0,84 - 0,86). Si, en cambio, observamos la diagonal que va de N=15 a N=17, podemos ver que la eficiencia continúa aumentando (0,68 - 0,80 - 0,88). Esto es incluso mejor que mantenerse constante, ya que significa que la utilización de los recursos no solo no disminuye sino que mejora. Ambos casos son, entonces, débilmente escalables.

3.4. Análisis de balance de carga

El balance de carga se calculó según la relación:

$$B = \frac{\text{Promedio } (T)}{\text{Máximo } (T)}$$

donde Promedio (T) representa el promedio de los tiempos de ejecución de los hilos y Máximo (T) representa el tiempo del hilo que más tardó. Esta métrica permite evaluar qué tan equitativamente se distribuyó el trabajo entre las unidades de procesamiento. Un valor cercano a 1 indica una distribución equilibrada, mientras que valores más bajos indican mayor desbalance.

Balance de carga	CARGA (N)				
UP	14	15	16	17	18
1	1	1	1	1	1
4	0.973	0.992	0.993	0.993	0.993
8	0.954	0.976	0.980	0.989	0.987
16	0.770	0.938	0.962	0.965	0.969

Los resultados muestran que el balance de carga es alto en la mayoría de las ejecuciones. Para 4 unidades de procesamiento, el balance se mantiene siempre por encima de 0.97, lo que indica una distribución muy pareja del trabajo. Para 8 unidades de procesamiento también se observa un buen comportamiento, con valores superiores a 0.95 y cercanos a 0.99 en las cargas más grandes.

El caso más desfavorable aparece con 16 unidades de procesamiento y $N=14$, donde el balance baja a 0.770. Esto se explica porque la carga de trabajo es pequeña en relación con la cantidad de hilos disponibles: al haber más unidades de procesamiento, el costo de coordinación y la irregularidad de los subárboles del backtracking tienen mayor peso relativo. Sin embargo, a medida que aumenta N , el balance mejora progresivamente hasta alcanzar 0.969 para $N=18$. Esto confirma que la estrategia de distribución dinámica de tareas resulta adecuada para este problema, ya que permite compensar parcialmente la diferencia de costo entre subárboles y mantener ocupadas las unidades de procesamiento cuando la carga de trabajo es suficientemente grande.

4. Uso de inteligencia artificial

Herramienta	PROMPT	ÉXITO	OBSERVACIÓN
ChatGPT	Comparar una asignación estática y una asignación dinámica de tareas para el problema de N-Reinas, considerando que el árbol de búsqueda es irregular y que los subárboles pueden tener costos distintos.	Sí	La comparación permitió justificar el uso de una asignación dinámica, ya que la carga de trabajo no es homogénea entre subárboles y una distribución fija podía provocar desbalance.
ChatGPT	Diseñar una estructura master-worker MPI+Pthreads para N-Reinas donde el proceso maestro genere tareas y los procesos trabajadores soliciten trabajo dinámicamente para luego resolverlo en sus hilos.	Parcial	Generó la estructura básica del programa MPI basada en un pool de tareas, pero lo hizo sin modularizar separando las funciones MPI de Pthreads.
Gemini	Generar el código paralelo para N-Reinas siguiendo la estructura dada.	Parcial	Dio una versión con asignación de memoria dinámica que interfería con el registro del tiempo, ya que se contaba esa asignación en el tiempo de ejecución.

Herramienta	PROMPT	ÉXITO	OBSERVACIÓN
Gemini	Reescribir la solución modularizando MPI y Pthreads en archivos separados.	Parcial	Desacopló la lógica de comunicación de red de la lógica de distribución interna de tareas, pero dejó un hilo en el proceso principal de MPI.
Gemini	Reestructuración para eliminar el uso de hilos en el archivo MPI.	Sí	Modificó correctamente la solución.
Gemini	Agregar un parámetro T para recibir la cantidad de hilos por nodo por consola y modificar el main de MPI para que use funciones contenedoras (f0, fN).	Sí	Modificó correctamente la solución.
ChatGPT	Incorporar una forma de calcular el balance de carga usando los tiempos de los hilos, interpretando el cociente entre tiempo promedio y tiempo máximo.	Sí	Permitió incorporar el calculo de balance de carga
ChatGPT	A partir de esta tabla de tiempos de ejecución, calcula los speedup (T_s/T_p) y eficiencia (S/P) y dámelas en el mismo formato.	Sí	Realizó una tarea que habría sido repetitiva y la devolvió en el formato exacto para copiar y pegar en la herramienta de redacción.